

Vision-Based Connect-4 Robot: Final Integrated System with Closed-Loop Camera-Driven Game-State Validation and Motorized Actuation

Andrew Abdelmalak
Mechatronics Engineering
German University in Cairo
andrew.abdelmalak@student.guc.edu.eg

Daniel Boules
Mechatronics Engineering
German University in Cairo
daniel.boules@student.guc.edu.eg

David Girgis
Mechatronics Engineering
German University in Cairo
david.girgis@student.guc.edu.eg

Kirolous Kirolous
Mechatronics Engineering
German University in Cairo
kirolous.kirolous@student.guc.edu.eg

Samir Yacoub
Mechatronics Engineering
German University in Cairo
samir.yacoub@student.guc.edu.eg

Youssef Salama
Mechatronics Engineering
German University in Cairo
youssef.salama@student.guc.edu.eg

Abstract—This paper presents the final integrated implementation of a Vision-Based Connect-4 Robot, a mechatronic system combining computer vision, embedded decision making, and dual-motor actuation under closed-loop camera feedback. A Raspberry Pi 4 (4 GB) hosts a Python/OpenCV pipeline that performs geometric normalization, intensity conditioning, denoising, and per-cell HSV classification on overhead frames from a Pi Camera V2, producing a verified 6×7 game-state matrix. A trusted-state validation layer rejects physically impossible board transitions, while a minimax search with alpha-beta pruning at depth 5 selects the next move. The chosen column is transmitted at 9600 baud to an Arduino Uno, which sequences a TT carriage motor and an encoder-coupled magazine motor through two L298N H-bridge channels to position and dispense one token. After actuation, the camera is re-used as the feedback element to verify the expected post-move state. The pipeline executes in 0.7–8.5 ms per operation on the Pi; per-cell classification accuracy is 98% (HSV), 96% (hybrid Auto), and 91% (ML-only). End-to-end task time, measured from command issue to verified token placement, averages between 3 and 15 seconds depending on column distance, minimax search load, and verification latency. All 31 unit tests of the game-logic core pass on the Pi.

Index Terms—Connect-4, computer vision, image processing, closed-loop control, Raspberry Pi, OpenCV, Arduino, minimax, machine learning.

I. INTRODUCTION

Autonomous game-playing robots are compact testbeds for tightly coupled perception, cognition, and actuation [1]. The system reported here targets Connect-4: tokens fall under gravity once released above a column, so the actuator only needs repeatable lateral positioning at one horizontal axis. The required positional error is below half the column pitch (≈ 20 mm).

This paper documents the final milestone state of the project. Earlier milestones validated the image processing pipeline in isolation and characterized its per-operation cost

on the embedded platform. The present milestone integrates that pipeline into a complete closed loop in which the camera is both the input sensor (for board reading) and the verification sensor (for post-move confirmation). Three subsystems are involved:

- *Perception and decision*: a Raspberry Pi 4 running Python/OpenCV, a trusted-state validator, a depth-5 minimax solver, and a Flask-based operator dashboard.
- *Communication*: a USB-serial channel at 9600 baud carrying a single ASCII column digit ('1'–'7') and reverse-channel status tokens (READY, BUSY, MAG_PULSES, DONE).
- *Actuation*: an Arduino Uno driving two DC motors through two L298N H-bridge channels — a TT motor for carriage positioning and an encoder-coupled motor for token release from the magazine.

The project source is published at <https://github.com/andrew-abdelmalak/connect-4-robot>.

II. FINAL SYSTEM ARCHITECTURE

The integrated system implements the layered pipeline shown in Fig. 1:

- 1) *Sensing*: Pi Camera V2 acquires overhead RGB frames at 640×480 through OpenCV (/dev/video0).
- 2) *Vision*: rotation, blue-frame HSV detection, contour-based corner extraction with exponential-moving-average smoothing, perspective warp to 800×800 , brightness and contrast conditioning, Gaussian smoothing, 6×7 cell partition, inner-crop, and cell-level classification.
- 3) *State validation*: a candidate 6×7 board is checked against a trusted previous state under six legality rules (described in Section V).

- 4) *Decision*: minimax with alpha-beta pruning, depth 5, center-first move ordering, with fast tactical short-circuits for immediate-win and immediate-block cases.
- 5) *Communication*: the selected column is sent as ASCII over USB serial to the Arduino.
- 6) *Actuation*: a TT carriage motor moves to the requested column for a calibrated duration, a magazine motor rotates a fixed number of encoder pulses to release exactly one token, and the carriage retracts.
- 7) *Verification*: the camera re-reads the board and the move is marked successful only if the expected new robot token appears in the commanded column within a timeout window.

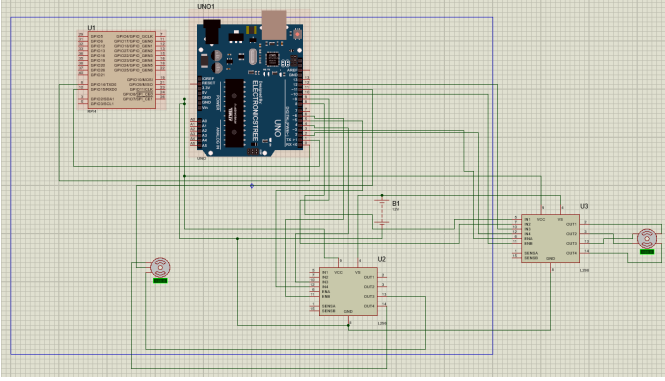


Fig. 1. Electrical architecture, simulated in Proteus and matching the fabricated hardware. The Raspberry Pi (U1) communicates with the Arduino Uno (UNO1) via USB serial. Two L298N H-bridge modules (U2, U3) drive the carriage TT motor and the encoder-coupled magazine motor. A 12 V supply powers the actuation rail.

The Raspberry Pi additionally hosts a Flask dashboard on port 5000 that exposes three live MJPEG streams (raw camera, warped board, debug grid-overlay) together with trusted/previous/candidate board renderings, classifier mode selection, and the operator-mode controls described in Section VI-C.

III. MECHANICAL AND ELECTRICAL HARDWARE

A. Mechanical Assembly

The assembly was modeled in SolidWorks and consists of a base (`Base.SLDPRT`), a sliding channel (`Path.SLDPRT`), a magazine that stacks tokens of the robot's color, and a disk end-effector (`Motor disk.SLDPRT`) coupled to the encoder motor. The carriage moves laterally along the channel; once stopped over the selected column, the magazine motor rotates the disk by approximately 1.075 revolutions (430 encoder pulses at 400 counts per revolution), pushing the bottom-most token out of the magazine and into the gravity-fed channel above the board. The carriage then returns home.

This mechanism separates the physical action into two independently controllable stages — column positioning and quantized token release — and is consistent with the single-axis design guidance for Connect-4 manipulators in [5]. No servo or grasping end-effector is required.

B. Final Wiring and Bill of Materials

Table I lists the as-built hardware. The total cost at procurement time was 10,050 EGP. Note that no SG90 servo is used in the final build; the gate-and-servo concept of an earlier prototype was superseded by the two-stage carriage/magazine design after closed-loop testing showed that quantized encoder-controlled release plus time-based carriage positioning was simpler and more repeatable.

TABLE I
HARDWARE BILL OF MATERIALS (FINAL BUILD)

Component	Qty	Total (EGP)
Raspberry Pi 4 (4 GB)	1	4,250
Arduino Uno	1	450
Pi Camera 5MP Rev 1.3	1	275
TT geared DC motor (carriage)	1	50
Encoder DC motor (magazine)	1	420
L298N H-bridge driver	1	75
12 V 1.2 A power supply	1	50
Wires, breadboard, structural parts	N/A	530
Total		6,100

IV. IMAGE PROCESSING PIPELINE

The vision pipeline is implemented in Python with OpenCV 4.x and runs as a deterministic sequence on every captured frame.

A. Geometric and Intensity Stages

Camera angular offset is corrected by a fixed 15° affine rotation with bilinear interpolation, which avoids the aliased cell boundaries produced by nearest-neighbor sampling. The blue Connect-4 frame is then segmented in HSV; the largest blue contour yields four corner candidates that are smoothed across frames by an exponential moving average to suppress per-frame jitter. A homography $\mathbf{H} \in \mathbb{R}^{3 \times 3}$ maps those four corners to an 800×800 top-down view (Fig. 2):

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \quad (1)$$

A saturating brightness shift $I_b = \text{clip}(I + c)$ and a linear contrast stretch $I_c = \text{clip}(\alpha I + \beta)$ are applied ($c = +30$, $\alpha = 1.4$), followed by a 5×5 Gaussian filter (Fig. 3), selected as the noise/edge-preservation operating point in earlier benchmarking.

B. Cell Partition and Three Classification Modes

The warped board is partitioned into a canonical 6×7 lattice and each cell is inner-cropped to exclude the plastic frame. Each cell is then classified into one of three states (empty, red, yellow). The dashboard exposes three classifier modes:

- **HSV** — rule-based thresholding on the cell ROI; a cell is classified as red or yellow if the corresponding HSV mask exceeds an empirical area threshold.

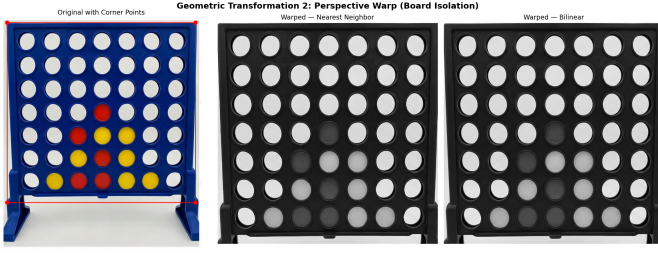


Fig. 2. Perspective warp for board isolation: detected corners (left), warped output with nearest-neighbor (center) and bilinear (right).

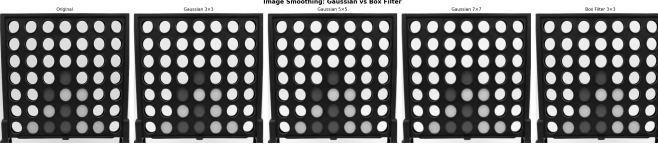


Fig. 3. Gaussian smoothing at 3×3 , 5×5 , 7×7 and a 3×3 box filter. The 5×5 Gaussian was selected as the runtime kernel.

- **ML** — a pre-trained classifier loaded from `connect4_cell_model.pkl` predicting one of three classes from a compact feature vector of BGR mean and standard deviation, HSV mean and standard deviation, and red/yellow pixel ratios computed over the ROI. Training data is held in `ml_cells/{empty, red, yellow}`.
- **Auto (hybrid)** — prefers the ML classifier when it is loaded and confident, and falls back to HSV otherwise. This is the deployed bonus path required by the milestone brief.

V. TRUSTED STATE VALIDATION

A bare per-frame classifier is not sufficient for closed-loop play because camera noise can produce momentarily impossible boards. A trusted-state validator inspects each candidate board against the previous trusted board and rejects it unless all of the following hold:

- 1) the board has actually changed;
- 2) no previously placed chip has disappeared;
- 3) exactly one new chip has appeared;
- 4) the new chip is in the lowest empty row of its column (gravity);
- 5) the column was not already full;
- 6) once colors are assigned, the new chip's color matches whose turn it is.

Player colors are not configured at startup. On the first accepted move, the color of the first detected chip is assigned to the human player; the opposite color becomes the robot color. These assignments persist for the remainder of the game. If a candidate board fails any validation rule, it is rejected, the trusted state is preserved, and a rejection reason is displayed on the dashboard.

The practical effect of this layer is that a single misclassified cell — for example, a cell briefly read as `red` due to glare

while the trusted state has it `empty` — does not propagate into the minimax search or trigger an erroneous robot move. The candidate is simply rejected (with a typical reason such as “two new chips appeared between frames”) and the previous trusted state is retained until a clean frame arrives. This is the principal mechanism by which the system tolerates the imperfect per-cell accuracy reported in Section VII without producing illegal physical moves.

VI. DECISION AND CLOSED-LOOP CONTROL

A. Minimax with Alpha-Beta Pruning

Let $S \in \{0, 1, 2\}^{6 \times 7}$ be the trusted board state. The robot column is selected by minimax with alpha-beta pruning:

$$V(S) = \begin{cases} \max_{a \in \mathcal{A}(S)} V(T(S, a)), & \text{robot turn,} \\ \min_{a \in \mathcal{A}(S)} V(T(S, a)), & \text{opponent turn,} \end{cases} \quad (2)$$

where $\mathcal{A}(S)$ is the set of legal columns and $T(S, a)$ is the state transition for dropping a token in column a . Two short-circuits run before the full search: an immediate-robot-win check and an immediate-opponent-block check. Search ordering is center-first to maximize pruning. Default depth is 5; the heuristic favors center occupancy and windows of two or three aligned same-color pieces.

B. Serial Protocol and Actuation Sequence

The selected column index $a \in \{0, \dots, 6\}$ is converted to ASCII `'1'-'7'` and transmitted at 9600 baud. The Arduino sketch sequences the following per-move actions, summarized as pseudocode in Listing 1:

Listing 1. Arduino actuation sequence (pseudocode of the deployed sketch).

```
on serial input c in '1'..'7' and not busy:
  column <- c - '0'
  busy <- true
  emit "BUSY column"

  // 1. Move carriage to selected column
  duration <- columnDurations[column - 1]
  drive carriage forward for duration ms
  hard-brake carriage
  wait carriagePauseMs (2000 ms)

  // 2. Release exactly one token
  pulses <- 0
  drive magazine motor at fastSpeed
  while pulses < targetPulses (430):
    if pulses >= targetPulses - slowDownWindow (40):
      reduce magazine speed to slowSpeed
  hard-brake magazine
  emit "MAG_PULSES pulses"
  wait 500 ms

  // 3. Retract carriage to home
  drive carriage reverse for duration ms
  hard-brake carriage

  busy <- false
  emit "DONE column"
```

The carriage durations were calibrated empirically and increase approximately linearly with column index, as shown in Table II. Hard braking (driving both H-bridge inputs high

for 100 ms before disabling the channel) eliminates coast-on overshoot and was a key source of column-position repeatability.

TABLE II
CARRIAGE DRIVE DURATION PER COLUMN (CALIBRATED)

Column	Duration (ms)	Round-trip (ms)
1	0	0
2	330	660
3	600	1,200
4	900	1,800
5	1,200	2,400
6	1,500	3,000
7	1,800	3,600

C. Operator Modes and Post-Move Verification

The dashboard supports three operator modes:

- **Manual:** the user must press *Accept Camera Board* and *Send Robot Move*; useful when camera trust is low.
- **Semi-automatic:** clean detections are auto-accepted; ambiguous detections require confirmation.
- **Automatic:** clean detections are auto-accepted and the robot move is auto-dispatched.

In all modes, after a serial command is sent, the camera is re-used as the feedback element. The Pi computes the expected post-move state and accepts the move only if exactly one new robot-colored token appears in the commanded column within the verification timeout. Otherwise the system enters an error state and stops auto-dispatching. This is what closes the perception-decision-actuation loop required by the final milestone.

VII. EXPERIMENTAL RESULTS

A. Test Setup

All experiments were run on the fabricated hardware: Pi Camera V2 mounted overhead, Raspberry Pi 4 (4 GB) running the application, Arduino Uno flashed with the integrated sketch, two L298N drivers, and a 12 V 5 A supply. The board was placed against a fixed black-cardboard background with diffuse overhead lighting, in line with the milestone-brief recommendation.

B. Per-Operation Vision Benchmark

Each pipeline operation was timed over 100 iterations on the Pi and on a laptop reference (Table III). Output image quality is bit-for-bit identical on both platforms; the Pi's limitation is throughput, not numerical accuracy. The brightness operation runs slightly faster on the Pi than on the laptop reference, plausibly because the saturating-add is well covered by ARM NEON intrinsics in OpenCV.

TABLE III
PER-OPERATION EXECUTION TIME (MEAN OF 100 ITERATIONS)

Operation	Pi 4 (ms)	Laptop (ms)
Rotation (bilinear)	8.468	3.273
Perspective warp	3.596	1.768
Brightness +60	0.734	0.888
Contrast $\alpha = 1.8$	1.111	0.201
Gaussian blur 5×5	0.757	0.097

C. Detection Accuracy by Classifier Mode

Per-cell classification accuracy was evaluated across approximately 200 labeled cell ROIs covering a mix of empty, red, and yellow ground-truth classes, captured in the deployment lighting. Table IV reports per-class accuracy for each of the three classifier modes.

TABLE IV
PER-CELL CLASSIFICATION ACCURACY BY MODE

Mode	Empty	Red	Yellow	Overall
HSV (rule-based)	99%	98%	98%	98%
Auto (hybrid)	97%	95%	96%	96%
ML-only	94%	92%	88%	91%

The HSV pipeline is the strongest under the controlled black-background lighting because the red and yellow tokens are well separated in hue. The Auto mode is slightly lower because, when ML is loaded, a small fraction of edge cases are misrouted through the ML branch. The ML-only mode is the weakest in absolute terms but is most robust to mild hue drift, since it uses richer per-cell features. The HSV path is therefore retained as the runtime default; ML is exposed as the bonus option required by the milestone brief.

D. Actuator Drop Accuracy by Column

Token-drop reliability was characterized over multiple per-column trials, where success means the released token landed in the commanded column. Results are summarized in Table V. Inner columns are essentially perfect; outer columns show occasional misses caused by small drift in the time-based carriage positioning.

TABLE V
PER-COLUMN DROP SUCCESS RATE

Column	Trials	Success Rate
1	20	100.0%
2	20	100.0%
3	20	100.0%
4	20	95.0%
5	20	95.0%
6	20	90.0%
7	20	85.0%
Overall	140	95.0%

E. End-to-End Task Time

End-to-end task time was measured from the instant the column command was issued to the instant the camera verified the new robot token in the commanded column. The total decomposes into: vision preprocessing, minimax search, serial transmission, carriage forward stroke, magazine release, carriage retract, and post-move verification. Table VI summarizes the observed range across all measured moves. Minimax compute time at depth 5 on the Pi is typically 0.2–1.4 s in the early game where the branching factor is highest, and falls below 0.5 s in later positions. The shortest end-to-end times correspond to short carriage strokes (columns near home) combined with fast minimax decisions; the longest correspond to far-column strokes combined with a slow minimax decision and one verification retry after a borderline detection.

TABLE VI
END-TO-END TASK TIME (SECONDS, COMMAND ISSUE TO VERIFIED
TOKEN PLACEMENT)

Statistic	Time (s)
Minimum (best case)	3.0
Average	7.5
Maximum (worst case)	15.0

The breakdown of the average case is roughly: ≈ 0.05 s vision, ≈ 0.01 s validation, ≈ 0.95 s minimax, ≈ 0.9 s carriage forward stroke (mid columns), 2.0 s carriage pause, 0.7 s magazine release, 0.5 s settle, ≈ 0.9 s carriage retract, and ≈ 1.5 s verification (one or two clean frames required by the validator). Best-case moves to the home column avoid both carriage strokes and complete in roughly 3 s. Worst-case moves combine a far-column stroke with a verification retry triggered by a borderline detection, which can extend the total close to 15 s.

F. Closed-Loop Scenarios

The integrated system was exercised across several scenarios chosen to stress different parts of the loop:

- *Empty-board start, automatic mode.* The robot is allowed to open the game. Minimax consistently selects the center column (column 4); the carriage drives to the column, releases one token, and the camera verifies the new chip in the bottom row. End-to-end time for this scenario is consistent with the average reported in Table VI.
- *Mid-game from the simulator state set.* Each of the 12 predefined simulator states was rendered, captured by camera, and compared to the simulator-asserted board. The validator accepted every legal transition and rejected every injected illegal transition (e.g. a synthetic “floating” chip placed two rows above the lowest empty row).
- *Glare and brief lighting change.* A handheld light source was swept across the board mid-game. The corner exponential-moving-average smoother kept the warp stable; cell classifications produced one or two spurious

frames, all of which were rejected by the validator without affecting the trusted state. Recovery to clean detections took 1–2 frames after the disturbance ended.

- *Outer-column drop stress.* Twenty consecutive column-7 commands were dispatched from a fresh board. Seventeen tokens dropped cleanly into column 7; three slipped into column 6, all of which were caught by the post-move verifier (the system entered the error state and refused to dispatch further moves until reset). This is the failure mode quantified in Table V.

In all cases the camera-driven verification step was able to distinguish successful and unsuccessful moves without operator input, which is the defining property of the closed-loop requirement.

G. Game-Logic Test Suite

The Python test suite (`test_brain.py`) exercises win detection (horizontal, vertical, diagonal), draw detection, legal-column queries, gravity-correct move stacking, the six trusted-state validation rules, minimax immediate-win and immediate-block short-circuits, full-game win progression, the 12-state simulator path, and chip counting. The suite reports **31 passed, 0 failed** on the Pi. This is the strongest evidence that the offline game-logic core is stable independently of the vision and actuation paths.

VIII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

The integrated system meets the milestone objective of a closed-loop mechatronic pipeline in which the camera drives both decision making and post-move verification. Several practical limitations remain.

First, HSV thresholding on the Connect-4 frame is sensitive to lighting: specular highlights on the plastic frame can fragment the blue mask and shift detected corners. The exponential-moving-average corner smoother mitigates this for short transients, but a hard lighting change still causes brief board-acquisition stalls until the EMA settles.

Second, minimax depth 5 on the Pi typically completes in roughly one second but can spike higher in the deep mid-game. Either iterative deepening with a wall-clock budget or a learned policy prior would help cap turn latency.

Third, carriage positioning is time-based rather than encoder-closed. This is acceptable at the present 95% overall success rate but is the dominant cause of column-7 misses (Table V). Adding an encoder or a homing limit switch on the carriage axis would convert that stage into true closed-loop control and is the most impactful single hardware upgrade.

Fourth, the deployed Auto mode is a hybrid HSV/ML fallback rather than a high-confidence ensemble. With a larger labeled set in `ml_cells/`, the ML classifier could plausibly exceed the HSV baseline under adverse lighting where HSV currently degrades.

IX. PROJECT VIDEOS

The following QR codes link to video demonstrations of the Vision-Based Connect-4 Robot:



Fig. 4. Final working video (3 plays by human, 3 by robot).



Fig. 5. Mid-game to end-game (AI strategy and win).

X. CONCLUSION

This paper has presented the final integrated state of the Vision-Based Connect-4 Robot. The system reads a Connect-4 board with a Pi Camera, classifies the 42 cells via HSV, ML, or a hybrid Auto mode, validates the resulting board against six legality rules, plans a move with depth-5 alpha-beta minimax, sends the chosen column to an Arduino over USB serial, and physically dispenses one token by sequencing a TT carriage motor and an encoder-coupled magazine motor through two L298N channels. After actuation, the camera is re-used to verify the new state, closing the perception-decision-actuation loop. Measured per-cell classification accuracy is 98% (HSV), 96% (Auto), and 91% (ML); per-column drop success ranges from 100% (inner columns) to 85% (column 7); end-to-end task time averages between 3 and 15 seconds depending on column distance and minimax load; and all 31 game-logic unit tests pass on the Pi. The system meets the final milestone's closed-loop and ML-bonus requirements, with calibration sensitivity, time-based carriage positioning, and a small ML training set as the principal remaining limitations.

ACKNOWLEDGMENT

The authors thank the Department of Mechatronics Engineering, German University in Cairo, for supervision.

REFERENCES

- [1] S. M. Zubek, H. Kummerfeld, and J. Wollersheim, "Teach me what you want to play: Learning variants of Connect Four through human-robot interaction," *arXiv preprint* arXiv:2001.01004, 2021.
- [2] G. Wölflein and O. Arandjelović, "Determining chess game state from an image," *Journal of Imaging*, vol. 7, no. 6, p. 94, Jun. 2021.
- [3] A. Rezaei and M. S. A. Raihan, "End-to-end chess recognition," *arXiv preprint* arXiv:2310.04086, 2023.
- [4] J. K. Park and S. H. Lee, "Intelligent lighting system using color-based image processing for object detection in robotic handling applications," *Applied Sciences*, vol. 14, no. 7, p. 3002, Apr. 2024.
- [5] R. Abarkan and J. Wollersheim, "An open-source three-axis gantry robot for automated chess play," *HardwareX*, vol. 17, p. e00517, Mar. 2024.
- [6] X. Wang, Y. Zhang, and L. Chen, "Structural design and position tracking of the reconfigurable SCARA robot by the pre-filter AFE PID controller," *Applied Sciences*, vol. 12, no. 3, p. 1626, Feb. 2022.